

Thermo-Fusion: Kamerabild und Wärmebild überlagert

Versuchs-Manuskript: ESP32-CAM + AMG8833 + Display-Board +
Browser-Dashboard — Aufbau, Programme, Zusammenspiel, alle Einstellungen

Prof. Dr.-Ing. Ralph Wystup M.Sc. — erstellt mit KI und Agent (Claude Code, Anthropic)

Stand: 3. August 2026, Kapitel 14 ergänzt am 29. September 2026

1. Zweck des Versuchs

Kommerzielle Wärmebildkameras blenden das Wärmebild halbtransparent über ein normales Kamerabild ein — so lassen sich Wärmequellen unmittelbar Objekten zuordnen („welche Klemme ist heiß?“, „wo verliert das Gehäuse Wärme?“). Dieser Versuch baut genau diese **Bildfusion** aus einfachen Bastelkomponenten nach:

- eine **ESP32-CAM** liefert das Live-Videobild,
- ein **AMG8833 Grid-EYE** (8×8-Infrarot-Array) liefert das Wärmebild,
- ein **Browser-Dashboard** legt beide Bilder übereinander — verschiebbar, skalierbar, spiegelbar und teiltransparent, bis sie deckungsgleich sind; die gefundene Ausrichtung wird dauerhaft gespeichert.

Daneben demonstriert der Versuch zwei übertragbare Ingenieur-Lektionen:

1. **Arbeitsteilige Architektur:** Jedes Gerät macht genau das, was es nachweislich gut kann — die Kamera streamt, das Display-Board misst und zeigt an, der Browser rechnet die Fusion. Alle Verbindungen laufen über einfaches HTTP im Heimnetz.
2. **Umgehen statt erzwingen:** Der ursprüngliche Plan (Sensor direkt an der ESP32-CAM) scheiterte an einer Hardware-Eigenheit des CAM-Boards (Abschnitt 10). Statt dagegen anzukämpfen, wurde die Architektur so umgestellt, dass nur nachweislich funktionierende Hardware-Software-Paare zum Einsatz kommen.

2. Versuchsaufbau

2.1 Komponenten

Komponente	Details
ESP32-Display-Board („CYD“-Typ)	ESP32 mit fest verbautem ILI9341-TFT (320×240, SPI); MicroPython
AMG8833 Grid-EYE (Adafruit-Breakout)	8×8-IR-Array, Blickfeld 60°, Auflösung 0,25 °C, I2C (Adresse 0x69), Versorgung 3,0-3,6 V , ~4,5 mA
AI-Thinker ESP32-CAM + MB-Board	OV2640-Kamera (VGA), WLAN; Versorgung 5 V über USB
PC im selben Heimnetz	Python (Standardbibliothek) + Browser

2.2 Verdrahtung

Der AMG8833 hängt am **Display-Board** (dort läuft er nachweislich):



Abbildung 1: Gesamtaufbau: links das Display-Board mit laufendem Wärmebild (Eisenpalette, Temperaturskala rechts, Datum/Uhrzeit unten), rechts die ESP32-CAM mit dem direkt daneben montierten AMG8833. Im Hintergrund ein Motor als warmes Testobjekt.

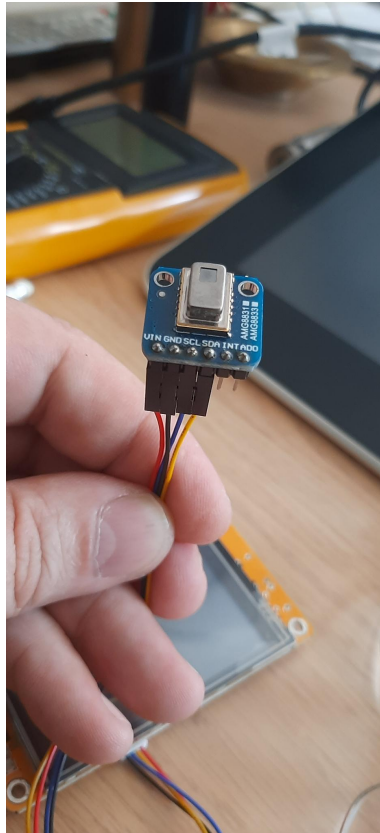


Abbildung 2: Der AMG8833 auf Adafruit-Breakout: Anschlüsse VIN, GND, SCL, SDA (INT und ADO bleiben frei).

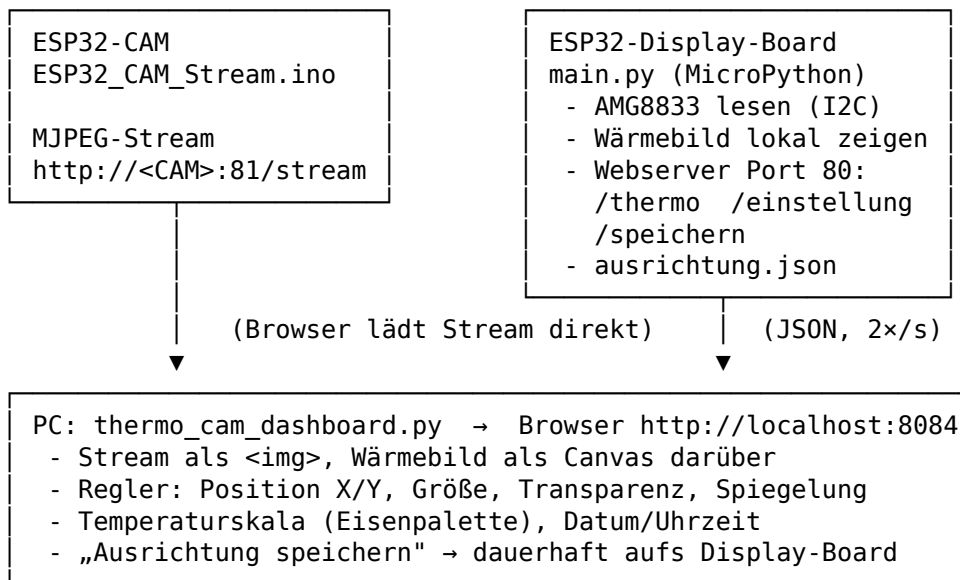
AMG8833	Display-Board
VIN	3,3 V
GND	GND
SCL	GPIO22
SDA	GPIO27
INT, ADO	frei

Die ESP32-CAM braucht keinerlei Zusatzverdrahtung — nur USB-Versorgung über ihr MB-Board. **Wichtig für die Fusion ist die Mechanik:** Der AMG8833 ist unmittelbar neben dem Kameraobjektiv montiert und blickt in dieselbe Richtung (siehe Foto). Je kleiner der Versatz der beiden „Augen“, desto besser gelingt die Deckung — den Rest erledigen die Ausrichtregler im Dashboard.

Interne Verdrahtung des Display-Boards (fest, zur Dokumentation): Display an SPI2 (SCK 14, MOSI 13, CS 15, DC 2, RST 4, Hintergrundbeleuchtung 21).

3. Architektur und Datenflüsse

Alles läuft über HTTP im Heimnetz — bewusst ohne Feldbus, Livebild und Livedaten direkt („HTTP liefert“):



Die drei Datenflüsse im Einzelnen:

1. **Videostream:** Der Browser lädt den MJPEG-Stream **direkt** von der Kamera (Port 81) — das Dashboard-Programm reicht nur die URL durch. Dadurch volle Bildrate ohne Umweg.
2. **Wärmedaten:** Das Dashboard-Programm fragt zweimal pro Sekunde `GET /thermo` beim Display-Board an und erhält JSON: die 64 Temperaturen in °C plus die aktuelle Ausrichtung.
3. **Ausrichtung:** Reglerbewegungen im Browser gehen (entprellt) als `GET /einstellung?ox=..&oy=..&sk=.` über das Dashboard-Programm ans Display-Board; `GET /speichern` legt sie dort dauerhaft in der Datei `ausrichtung.json` ab. Beim nächsten Start lädt das Board sie selbst, und das Dashboard übernimmt sie in die Regler — einmal ausrichten genügt also.

4. Der Sensor AMG8833 (Grid-EYE)

Panasonic-Infrarot-Array mit $8 \times 8 = 64$ Thermoelementen; jedes Pixel liefert die Temperatur der Fläche, die es „sieht“ (Blickfeld insgesamt $60^\circ \times 60^\circ$). Auslesung per I2C (Adresse 0x69, alternativ 0x68 über den ADO-Pin): 128 Bytes ab Register 0x80, je Pixel ein 12-Bit-Wert im Zweierkomplement mit **0,25 °C je Digit** — die Umrechnung ist also eine Multiplikation. Bildrate intern 10 Hz. Versorgung strikt 3,0–3,6 V ($\sim 4,5$ mA); das Adafruit-Breakout verträgt am VIN auch 5 V (eigener Regler), das nackte Bauteil nicht.

5. Programm 1: main.py (Display-Board, MicroPython)

Das umfangreichste Programm des Versuchs. Es vereint vier Aufgaben: lokale Wärmebildanzeige, Sensor-Auslesung, Webserver und Ausrichtungs-Speicher.

5.1 Einstellungen (Dateikopf)

Konstante	Vorgabe	Bedeutung
DYNAMISCHE_FARBSKALA	True	Skala folgt geglättet den Messwerten; False = feste Grenzen
FESTE_MIN_TEMP / FESTE_MAX_TEMP	20 / 36	Grenzen bei fester Skala (°C)
MIN_SPANNE	4.0	kleinste Skalen-Spreizung (K) — verhindert Rausch-Konfetti bei gleichmäßig warmem Bild
GLAETTUNG	0.3	Trägheit der dynamischen Skala (0 = starr, größer = nervöser)
FARBSKALA_ANZEIGEN / SKALA_BREITE	True / 40	Farbbalken rechts ein/aus, Streifenbreite in Pixeln
FELD_PIXEL	8	Kachelgröße des Wärmebilds in Pixeln
FARBPALETTE	„eisen“	„eisen“ (schwarz→violett→rot→gelb→weiß) oder „regenbogen“
UHR_ANZEIGEN	True	Datum/Uhrzeit-Leiste unten
WLAN_SSID / WLAN_PASS	(eintragen!)	Heimnetz-Zugang — nötig für Webserver und NTP-Uhr
ZEITZONE_OFFSET_H	2	Sommerzeit 2, Winterzeit 1
HTTP_PORT	80	Port des Mini-Webservers

5.2 Funktionsweise

- **Start:** WLAN verbinden (bis 15 s), Uhr per NTP stellen, Webserver starten — und die **IP-Adresse 5 Sekunden groß auf dem Display anzeigen** (DISPLAY_IP = "..."), damit sie ohne Thonny ins Dashboard übernommen werden kann. Ohne WLAN läuft das Gerät als reine Lokalanzeige weiter.
- **Wärmebild:** Die 64 Sensorwerte werden bilinear auf 35×28 Kacheln (à 8×8 Pixel) interpoliert und über eine vorberechnete 256-Stufen-Eisenpalette eingefärbt. Die Ausgabe erfolgt zeilenweise als Pixel-Bänder in einem SPI-Transfer je Kachelzeile (schnell, flackerfrei; SPI-Takt 20 MHz — 40 MHz lehnt die MicroPython-Firmware ab).

- **Dynamische Skala:** Min/Max der Messung werden exponentiell geglättet (GLAETTUNG), eine Mindest-Spreizung (MIN_SPANNE) verhindert, dass Rauschen bei einheitlicher Szene zu wildem Farbflimmern wird. Der Farbbalken rechts trägt oben den Max-, unten den Min-Wert der Skala.
- **Uhrleiste:** Datum/Uhrzeit unten, Neuzeichnung nur beim Sekundenwechsel.
- **Mini-Webserver:** Nicht blockierender Socket-Server (Port 80), wird auch **während des Zeichnens** regelmäßig bedient, damit Dashboard-Anfragen nie warten müssen. Drei Endpunkte:

Endpunkt	Antwort/Aktion
/thermo	JSON {"t":[64 Temperaturen °C], "einst":{"ox,oy,sk,al,sp}}
/einstellung?ox=&oy=&sk=&al=&sp=	Ausrichtung setzen (mit Grenzen: sk 25-400 %, al 0-100 %, sp Bit 0/1)
/speichern	Ausrichtung in ausrichtung.json schreiben (Flash-Dateisystem, neustartfest)

6. Programm 2: ESP32_CAM_Stream.ino (ESP32-CAM, Arduino/C++)

Die Kamera ist bewusst auf ihre Kernaufgabe reduziert: Sie ist eine reine Netzwerkkamera. Der Sketch initialisiert die OV2640 (VGA 640×480, JPEG-Qualität 10, PSRAM-Doppelpuffer), verbindet sich mit dem WLAN und liefert einen **MJPEG-Livestream** über den eingebauten HTTP-Server:

- Stream: `http://<Kamera-IP>:81/stream` (Multipart-JPEG — jeder Browser zeigt ihn direkt an)
- Die IP meldet der serielle Monitor (115200 Baud) beim Start; in der Fritzbox „immer dieselbe IP zuweisen“ ankreuzen.
- Versorgung: 5 V über das MB-Board/USB — ein kräftiges Kabel und ein kräftiger Port sind bei diesem Board wichtig (Brownout-empfindlich).

Alternativ tut es der funktionsgleiche `ESP32_CAM_Thermo_Fusion.ino` (er streamt identisch und ignoriert den fehlenden Sensor) — für den Regelbetrieb ist der schlichte Stream-Sketch die sauberere Wahl: so wenig Software wie nötig auf jedem Gerät.

7. Programm 3: thermo_cam_dashboard.py (PC, Python)

Reines Python ohne Zusatzpakete; startet einen lokalen Webserver und öffnet die Bedienoberfläche im Browser.

7.1 Einstellungen (Dateikopf)

Konstante	Vorgabe	Bedeutung
ESP32CAM_IP	(eintragen)	IP der Kamera — nur für die Stream-URL
DISPLAY_IP	(eintragen)	IP des Display-Boards — Wärmedaten und Ausrichtung
STREAM_PORT	81	Streamport der Kamera
THERMO_INTERVALL	0.5	Sekunden zwischen zwei Wärmedaten-Abfragen
WEB_PORT	8084	Port der Bedienoberfläche (<code>http://localhost:8084</code>)

7.2 Die Überlagerung im Browser

Der eigentliche Fusions-Trick ist bewusst einfach und nutzt den Browser als Grafikprozessor:

- Der **Stream** liegt als -Element in einer 640×480-Bühne.
- Das **Wärmebild** ist ein winziges 8×8-Pixel-Canvas, das je Aktualisierung aus den 64 Temperaturen neu gefüllt wird (Eisenpalette, geglättete Min/Max-Skala wie am Display-Gerät). Dieses Canvas wird per CSS auf die eingestellte Größe skaliert — **die weiche bilineare Interpolation erledigt der Browser** — und mit CSS-Eigenschaften positioniert (left/top = Offset), gespiegelt (transform: scale) und teiltransparent gemacht (opacity).
- Ergebnis: Die Überlagerung reagiert verzögerungsfrei auf jede Reglerbewegung, ohne dass ein einziges Pixel im PC-Programm gerechnet wird.

7.3 Bedienelemente

Regler	Bereich	Wirkung
Position X / Y	±250 px	Wärmebild verschieben
Größe	25-400 %	Wärmebild skalieren (100 % = Bildbreite; Sensor ist quadratisch)
Transparenz	0-100 %	Sichtbarkeit des Wärmebilds (45 % = guter Start)
horizontal/vertikal spiegeln	an/aus	Einbaulage des Sensors ausgleichen
„Ausrichtung speichern“	—	Einstellung dauerhaft aufs Display-Board

Dazu: Statuspunkt (Verbindung zum Display-Board), Temperaturskala mit Max/Min (geglättet), Datum/Uhrzeit, Fehlerband bei Störungen. Beim ersten Verbinden übernimmt das Dashboard die auf dem Board gespeicherte Ausrichtung in die Regler.

8. Ausrichtung: die Bilder deckungsgleich bringen

1. Kamera und Sensor **mechanisch fixieren** (nebeneinander, gleiche Blickrichtung) — jede spätere Verschiebung der Mechanik macht die gespeicherte Ausrichtung ungültig.
2. Ein **warmes, klar umrissenes Objekt** ins Bild bringen: Hand, Kaffeetasse, Lötkolben in sicherem Abstand.
3. Zuerst die **Spiegelungs-Häkchen** prüfen: Bewegt sich der warme Fleck entgegengesetzt zur Hand, spiegeln.
4. Mit **Größe** beginnen (das 60°-Feld des Sensors entspricht bei dieser Kamera grob 90-110 %), dann **Position X/Y** nachführen, bis der Fleck auf der Hand liegt. Transparenz nach Geschmack.
5. „**Ausrichtung speichern**“ — fertig. Die Einstellung übersteht Neustarts beider Geräte.

Physikalische Grenzen, ehrlich benannt: Kamera und Sensor sitzen wenige Zentimeter auseinander (**Parallaxe**) — eine Ausrichtung gilt daher streng nur für eine Entfernung; bei sehr nahen Objekten wandert die Deckung. Für Szenen ab etwa einem Meter ist der Fehler mit 8×8 Wärme-Pixeln praktisch unsichtbar. Auch unterscheiden sich die Blickfelder (Sensor 60° quadratisch, Kamera ~65° in 4:3) — Offset und Skalierung gleichen das im Rahmen der groben Wärmebild-Auflösung vollständig aus.

9. Inbetriebnahme Schritt für Schritt

1. **Display-Board:** In main.py WLAN-Zugangsdaten eintragen; Datei mit Thonny als main.py aufs Board. Neustart → das Display zeigt 5 s die Zeile `DISPLAY_IP = "..."`, danach das Wärmebild.
2. **ESP32-CAM:** ESP32_CAM_Stream.ino geflasht, USB-Versorgung. Kontrolle im Browser: `http://<CAM-IP>:81/stream` zeigt das Livebild.
3. **Fritzbox:** beiden Geräten „immer dieselbe IPv4-Adresse zuweisen“.
4. **PC:** In thermo_cam_dashboard.py beide IPs eintragen; python thermo_cam_dashboard.py; Browser auf `http://localhost:8084`.
5. Ausrichten nach Abschnitt 8, speichern.

Danach ist der Betrieb zweigleisig: Das **Display-Board** arbeitet autark als Wärmebildgerät (mit Skala und Uhr), das **Dashboard** zeigt zusätzlich die Fusion — beides gleichzeitig, beliebig ein- und ausschaltbar.

10. Die Vorgeschichte: warum der Sensor nicht an der Kamera hängt

Der erste Entwurf schloss den AMG8833 direkt an die freien Pins der ESP32-CAM an (GPIO13/14, SD-Bus). Es folgte eine lehrreiche Fehlersuche, deren Ergebnis hier festgehalten ist, weil es für alle ESP32-CAM-Projekte gilt:

- **Symptome:** Board bootet nicht bzw. startet im Sekundentakt neu (POWERON_RESET), sobald die I2C-Leitungen angeschlossen sind; Zeichensalat in der seriellen Ausgabe; Sensor wird nie gefunden — obwohl Sensor (Kreuztest am Display-Board), CAM-Pins (Drahtbrücken-Loopback-Test), Kabel (Tausch) und Versorgung einzeln nachweislich in Ordnung waren.
- **Ursache:** **GPIO14 ist die SD-Karten-Taktleitung des ESP32-CAM. Ein Pull-up-Widerstand an diesem Pin — wie ihn jedes I2C-Modul auf SDA/SCL trägt — verhindert den Boot.** (Dokumentiert u. a. im ESP32-Forum mit einem DS18B20 am selben Pin.) Auch nach Umzug des Taktes auf GPIO15 blieb die Kombination auf dieser Hardware unzuverlässig (Versorgungsempfindlichkeit des CAM-Boards).
- **Konsequenz:** Architekturwechsel statt Hardware-Kampf — der Sensor blieb am Display-Board, wo er läuft; die Kamera wurde zur reinen Netzwerkkamera. Das Ergebnis ist robuster und funktional reicher (lokale Anzeige + Fusion gleichzeitig).
- **Merkmale:** An der ESP32-CAM für I2C GPIO13 (SDA) und GPIO15 (SCL) verwenden, GPIO14 meiden; AMG8833 nie direkt an 5 V; die Diagnose-Sketches ESP32_CAM_I2C_Test.ino (Pin-Suchlauf) und ESP32_CAM_Pin_Loopback.ino (Brückentest) bleiben im Projektordner für künftige Fälle.

11. Fehlersuche im Betrieb

Symptom	Ursache / Abhilfe
Dashboard: „Keine Verbindung zum Display-Board“	DISPLAY_IP falsch/veraltet (Display-Neustart zeigt sie 5 s an); Board ohne WLAN (Startbildschirm „KEIN WLAN!“)
Kein Kamerabild	Stream-URL direkt testen (<code>http://<CAM>:81/stream</code>); CAM-Versorgung (kräftiges USB); nur ein Betrachter je Stream
Wärmebild ruckelt im Dashboard	normal ist 2 Hz (THERMO_INTERVALL); kleiner stellen belastet das Display-Board kaum

Symptom	Ursache / Abhilfe
Regler wirken nicht	Verbindung zum Display-Board prüfen (Statuspunkt); Browserseite neu laden
Ausrichtung nach Neustart weg	„Ausrichtung speichern“ wurde nicht gedrückt (Regler wirken sofort, gespeichert wird nur auf Knopfdruck)
Display zeigt Wärmebild, aber Dashboard nichts	PC und Boards im selben Netz?
Uhrzeit falsch	Firewall-Freigabe für Python? NTP beim Start ohne WLAN fehlgeschlagen; ZEITZONE_OFFSET_H prüfen (Sommer 2 / Winter 1)

12. Dateiübersicht

Datei	Gerät	Rolle
main.py	Display-Board	aktiv: Wärmebild + Webserver + Ausrichtungs-Speicher
ESP32_CAM_Stream.ino	ESP32-CAM	aktiv: MJPEG-Stream
thermo_cam_dashboard.py	PC	aktiv: Fusions-Dashboard (Port 8084)
ausrichtung.json	Display-Board (automatisch)	gespeicherte Ausrichtung
main_waermebild_v1.py / v2.py	Archiv	frühere Display-Stände (v1 = Original, v2 = ohne Webserver)
ESP32_CAM_Thermo_Fusion.ino	Archiv	Variante mit Sensor an der CAM (siehe Abschnitt 10)
ESP32_CAM_I2C_Test.ino,	Werkzeug	Diagnose-Sketches aus der Fehlersuche
ESP32_CAM_Pin_Loopback.ino		
VTermo1.jpg, VTermo2.jpg	Doku	Fotos des Versuchsaufbaus

13. Ausbaumöglichkeiten

- **Temperatur unterm Mauszeiger:** Das Dashboard kennt alle 64 Werte — ein Maus-Ereignis auf der Bühne könnte die Temperatur an der Zeigerposition einblenden.
- **Aufzeichnung:** Wärmedaten als CSV mitschreiben (Zeitreihe je Pixel) oder bei Grenzwertüberschreitung automatisch ein Standbild des Streams sichern.
- **Alarmierung:** Das Display-Board könnte bei Übertemperatur einen Endpunkt /alarm setzen oder die RGB-LED des Boards schalten.
- **Mehrere Sensoren:** Weitere Display-Boards (oder reine Sensor-Knoten) liefern zusätzliche /thermo-Quellen; das Dashboard könnte zwischen ihnen umschalten.
- **Bessere Sensorik:** Ein MLX90640 (32×24 Pixel) am selben Architekturmuster würde die Wärmebild-Auflösung verzehnfachen — Registerplan bzw. JSON-Format wachsen einfach mit.

14. Die Simulation in der Seite

Die Seite Waermebild_<Fassung>.html enthält den Versuch ohne Gerät: eine gezeichnete Szene, aus der die 64 Sensorwerte entstehen, das Display-Board mit dem Code von main.py und das Dashboard mit dem Code von thermo_cam_dashboard.py. Was am Gerät gerechnet wird, wird

hier mit denselben Zeilen gerechnet; nur die Szene und der Sensor sind Modell. Alle Zahlen sind aus der Geometrie abgeleitet und stehen hier zum Nachrechnen.

14.1 Die Szene

Vor einer Wand mit Umgebungstemperatur T_u steht ein Tisch ($T_u + 0,5$ K), darauf eine warme Tasse (T_t , Henkel $T_t - 6$ K, Henkelloch T_u) und ein kaltes Glas (T_g). Die Größen sind für 1 m festgelegt (Tasse 90×130 px im Kamerabild von 640×480 px) und skalieren mit $1/d$, wenn die Entfernung d verstellt wird. Die Temperaturkarte der Szene hat 160×120 Zellen à 4 px.

14.2 Der Sensor

Die Kamera bildet 65° in 4:3 auf 640 px ab; ihre Brennweite in Pixeln ist

$$f = \frac{320}{\tan 32,5^\circ} = 502,3 \text{ px.}$$

Der Sensor sieht 60° quadratisch; im Kamerabild sind das

$$b_s = 2f \tan 30^\circ = 580,0 \text{ px,} \quad \text{ein Sensorpixel} = 72,5 \text{ px.}$$

Deshalb liegt die richtige Größe im Dashboard bei $580/640 = 90,6\%$, was der Erfahrungswert „grob 90-110 %“ aus Abschnitt 8 bestätigt.

Die Sensorachse ist gegen die Kameraachse versetzt: um den Montagewinkel φ (in der Seite einstellbar, Vorgabe $1,5^\circ$) und um die Parallaxe aus dem Abstand b der beiden „Augen“ (Vorgabe 3 cm):

$$x_s = 320 + f \tan \varphi + \frac{fb}{d}.$$

Der Winkelanteil ist eine feste Verschiebung, die der Regler „Position X“ ein für alle Mal ausgleicht. Der Parallaxenanteil hängt von der Entfernung ab: bei $b = 3$ cm sind es 15 px bei 1 m, aber 50 px bei 0,3 m — das ist die in Abschnitt 8 benannte Grenze, und die Seite zeigt sie, wenn nach dem Speichern die Entfernung verstellt wird. Ein Sensorpixel misst 72,5 px; oberhalb von etwa einem Meter bleibt der Parallaxenfehler darunter.

Jedes der 64 Pixel sieht $7,5^\circ \times 7,5^\circ$ und mittelt über sein Feld (10×10 Stützstellen der Temperaturkarte; außerhalb des Kamerabilds gilt die Wandtemperatur). Dazu kommt gaußsches Rauschen mit einstellbarer Streuung (Vorgabe 0,25 K) und die Quantisierung auf 0,25 °C je Digit wie im 12-Bit-Wert des Bausteins. Ein um 180° gedrehter Einbau vertauscht Zeilen und Spalten — dann sind beide Spiegelungs-Häkchen nötig, wie in Abschnitt 8 beschrieben.

14.3 Display und Dashboard

Das Display-Board rechnet in der Seite mit den Funktionen aus main.py, Zeile für Zeile übertragen: `interpoliere_zeile` (bilinear $8 \times 8 \rightarrow 35 \times 28$), `baue_palette` (256 Stufen, Eisenpalette, RGB565), die geglättete Skala mit `GLAETTUNG = 0.3` und `MIN_SPANNE = 4.0`, der Farbbalken mit Max oben und Min unten, die Uhrzeile. Das Dashboard verwendet den `<script>`-Teil von `thermo_cam_dashboard.py` unverändert: `farbe`, `malen`, die Regler und deren Grenzen; „Ausrichtung speichern“ legt die Werte im Browser ab, wo am Gerät `ausrichtung.json` liegt.

Als Hilfe beim Ausrichten nennt die Seite die Deckung: den Schwerpunkt der Sensorpixel oberhalb der Mitte zwischen Min und Max, mit der eingestellten Lage in Bühnenkoordinaten gebracht und mit der Tassenmitte verglichen. Der Knopf „Rechnerische Ausrichtung übernehmen“ setzt die aus 14.2 folgenden Werte — für die eingestellte Entfernung.

14.4 Prüfung

Die Seite wird im echten Browser geprüft (seite/pruefe_seite.mjs, Ergebnisse in seite/PRUEFPLAN_Seite.md), keine Fehler beim Laden, Bedienelemente auf dem ersten Bildschirm, die Interpolation und die Paletten-Indizes gegen eine unabhängige Nachrechnung mit dem Python-Code aus main.py (seite/referenz_anzeige.py), Min und Max der Matrix gegen die eingestellten Temperaturen bei abgeschaltetem Rauschen, die Deckung nach rechnerischer Ausrichtung besser als ein Sensorpixel, die Parallaxe nach Entfernungswechsel, die Dokumentation in der Seite.